

本书(简称 CS:APP)的主要读者是计算机科学家、计算机工程师,以及那些想通过学习计算机系统的内在运作而能够写出更好程序的人。

我们的目的是解释所有计算机系统的本质概念,并向你展示这些概念是如何实实在在地影响应用程序的正确性、性能和实用性的。其他的系统类书籍都是从构建者的角度来写的,讲述如何实现硬件或系统软件,包括操作系统、编译器和网络接口。而本书是从程序员的角度来写的,讲述应用程序员如何能够利用系统知识来编写出更好的程序。当然,学习一个计算机系统应该做些什么,是学习如何构建一个计算机系统的很好的出发点,所以,对于希望继续学习系统软硬件实现的人来说,本书也是一本很有价值的介绍性读物。大多数系统书籍还倾向于重点关注系统的某一个方面,比如:硬件架构、操作系统、编译器或者网络。本书则以程序员的视角统一覆盖了上述所有方面的内容。

如果你研究和领会了这本书里的概念,你将开始成为极少数的“牛人”,这些“牛人”知道事情是如何运作的,也知道当事情出现故障时如何修复。你写的程序将能够更好地利用操作系统和系统软件提供的功能,对各种操作条件和运行时参数都能正确操作,运行起来更快,并能避免出现使程序容易受到网络攻击的缺陷。同时,你也要做好更深入探究的准备,研究像编译器、计算机体系结构、操作系统、嵌入式系统、网络互联和网络安全这样的高级题目。

读者应具备的背景知识

本书的重点是执行 x86-64 机器代码的系统。对英特尔及其竞争对手而言, x86-64 是他们自 1978 年起,以 8086 微处理器为代表,不断进化的最新成果。按照英特尔微处理器产品线的命名规则,这类微处理器俗称为“x86”。随着半导体技术的演进,单芯片上集成了更多的晶体管,这些处理器的计算能力和内存容量有了很大的增长。在这个过程中,它们从处理 16 位字,发展到引入 IA32 处理器处理 32 位字,再到最近的 x86-64 处理 64 位字。

我们考虑的是这些机器如何在 Linux 操作系统上运行 C 语言程序。Linux 是众多继承自最初由贝尔实验室开发的 Unix 的操作系统中的一种。这类操作系统的其他成员包括 Solaris、FreeBSD 和 MacOS X。近年来,

由于 Posix 和标准 Unix 规范的标准化努力，这些操作系统保持了高度兼容性。因此，本书内容几乎直接适用于这些“类 Unix”操作系统。

文中包含大量已在 Linux 系统上编译和运行过的程序示例。我们假设你能访问一台这样的机器，并且能够登录，做一些诸如切换目录之类的简单操作。如果你的计算机运行的是 Microsoft Windows 系统，我们建议你选择安装一个虚拟机环境(例如 VirtualBox 或者 VMWare)，以便为一种操作系统(客户 OS)编写的程序能在另一种系统(宿主 OS)上运行。

我们还假设你对 C 和 C++ 有一定的了解。如果你以前只有 Java 经验，那么你需要付出更多的努力来完成这种转换，不过我们也会帮助你。Java 和 C 有相似的语法和控制语句。不过，有一些 C 语言的特性(特别是指针、显式的动态内存分配和格式化 I/O)在 Java 中都是没有的。所幸的是，C 是一个较小的语言，在 Brian Kernighan 和 Dennis Ritchie 经典的“K&R”文献中得到了清晰优美的描述[61]。无论你的编程背景如何，都应该考虑将 K&R 作为个人系统藏书的一部分。如果你只有使用解释性语言的经验，如 Python、Ruby 或 Perl，那么在使用本书之前，需要花费一些时间来学习 C。

本书的前几章揭示了 C 语言程序和它们相对应的机器语言程序之间的交互作用。机器语言示例都是用运行在 x86-64 处理器上的 GNU GCC 编译器生成的。我们不需要你以前有任何硬件、机器语言或是汇编语言编程的经验。

给 C 语言初学者 关于 C 编程语言的建议

为了帮助 C 语言编程背景薄弱(或全无背景)的读者，我们在书中加入了这样一些专门的注释来突出 C 中一些特别重要的特性。我们假设你熟悉 C++ 或 Java。

如何阅读此书

从程序员的角度学习计算机系统是如何工作的会非常有趣，主要是因为你可以主动地做这件事情。无论何时你学到一些新的东西，都可以马上试验并且直接看到运行结果。事实上，我们相信学习系统的唯一方法就是做(do)系统，即在真正的系统上解决具体的问题，或是编写和运行程序。

这个主题观念贯穿全书。当引入一个新概念时，将会有有一个或多个练习题紧随其后，你应该马上做一做来检验你的理解。这些练习题的解答在每章的末尾。当你阅读时，尝试自己来解答每个问题，然后再查阅答案，看自己的答案是否正确。除第 1 章外，每章后面都有难度不同的家庭作业。对每个家庭作业题，我们标注了难度级别：

- 只需要几分钟。几乎或完全不需要编程。

•• 可能需要将近 20 分钟。通常包括编写和测试一些代码。(许多都源自我们在考试中出的题目。)

•• 需要很大的努力，也许是 1~2 个小时。一般包括编写和测试大量的代码。

•• 一个实验作业，需要将近 10 个小时。

文中每段代码示例都是由经过 GCC 编译的 C 程序直接生成并在 Linux 系统上进行了测试，没有任何人为的改动。当然，你的系统上 GCC 的版本可能不同，或者根本就是另外一种编译器，那么可能生成不一样的机器代码，但是整体行为表现应该是一样的。所有的源程序代码都可以从 csapp.cs.cmu.edu 上的 CS:APP 主页上获取。在本书中，源程序的文件名列在两条水平线的右边，水平线之间是格式化的代码。比如，图 1 中的程序能在 code/intro/ 目录下的 hello.c 文件中找到。当遇到这些示例程序时，我们鼓励你在自己的系统上试着运行它们。

```

1  #include <stdio.h>
2
3  int main()
4  {
5      printf("hello, world\n");
6      return 0;
7  }
```

图 1 一个典型的代码示例

为了避免本书体积过大、内容过多，我们添加了许多网络旁注(Web aside)，包括一些对本书主要内容的补充资料。本书中用 CHAP:TOP 这样的标记形式来引用这些旁注，这里 CHAP 是该章主题的缩写编码，而 TOP 是涉及的话题的缩写编码。例如，网络旁注 DATA:BOOL 包含对第 2 章中数据表示里面有关布尔代数内容的补充资料；而网络旁注 ARCH:VLOG 包含的是用 Verilog 硬件描述语言进行处理器设计的资料，是对第 4 章中处理器设计部分的补充。所有的网络旁注都可以从 CS:APP 的主页上获取。

旁注 什么是旁注

在整本书中，你将会遇到很多以这种形式出现的旁注。旁注是附加说明，能使你对当前讨论的主题多一些了解。旁注可以有很多用处。一些是小的历史故事。例如，C 语言、Linux 和 Internet 是从何而来的？有些旁注则是用来澄清学生们经常感到疑惑的问题。例如，高速缓存的行、组和块有什么区别？还有些旁注给出了一些现实世界的例子。例如，一个浮点错误怎么毁掉了法国的一枚火箭，或是给出市面上出售的一个磁盘驱动器的几何和运行参数。最后，还有一些旁注仅仅就是一些有趣的内容，例如，什么是“hoinky”？

本书概述

本书由 12 章组成，旨在阐述计算机系统的核心概念。内容概述如下：

- 第 1 章：计算机系统漫游。这一章通过研究“hello, world”这个简单程序的生命周期，介绍计算机系统的主要概念和主题。
- 第 2 章：信息的表示和处理。我们讲述了计算机的算术运算，重点描述了会对程序员有影响的无符号数和数的补码表示的特性。我们考虑数字是如何表示的，以及由此确定对于一个给定的字长，其可能编码值的范围。我们探讨有符号和无符号数字之间类型转换的效果，还阐述算术运算的数学特性。菜鸟级程序员经常很惊奇地了解到(用补码表示的)两个正数的和或者积可能为负。另一方面，补码的算术运算满足很多整数运算的代数特性，因此，编译器可以很安全地把一个常量乘法转化为一系列的移位和加法。我们用 C 语言的位级操作来说明布尔代数的原理和应用。我们从两个方面讲述了 IEEE 标准的浮点格式：一是如何用它来表示数值，一是浮点运算的数学属性。

对计算机的算术运算有深刻的理解是写出可靠程序的关键。比如，程序员和编译器不能用表达式 $(x-y < 0)$ 来替代 $(x < y)$ ，因为前者可能会产生溢出。甚至也不能用表达式 $(-y < -x)$ 来替代，因为在补码表示中负数和正数的范围是不对称的。算术溢出是造成程序错误和安全漏洞的一个常见根源，然而很少有书从程序员的角度来讲述计算机算术运算的特性。

- 第 3 章：程序的机器级表示。我们教读者如何阅读由 C 编译器生成的 x86-64 机器代码。我们说明为不同控制结构(比如条件、循环和开关语句)生成的基本指令模式。我们还讲述过程的实现，包括栈分配、寄存器使用惯例和参数传递。我们讨论不同数据结构(如结构、联合和数组)的分配和访问方式。我们还说明实现整数和浮点数算术运算的指令。我们还以分析程序在机器级的样子作为途径，来理解常见的代码安全漏洞(例如缓冲区溢出)，以及理解程序员、编译器和操作系统可以采取的减轻这些威胁的措施。学习本章的概念能够帮助读者成为更好的程序员，因为你们懂得程序在机器上是如何表示的。另外一个好处就在于读者会对指针有非常全面而具体的理解。
- 第 4 章：处理器体系结构。这一章讲述基本的组合和时序逻辑元素，并展示这些元素如何在数据通路中组合到一起，来执行 x86-64 指令集的一个称为“Y86-64”的简化子集。我们从设计单时钟周期数据通路开始。这个设计概念上非常简单，但是运行速度不会太快。然后我们引入流水线的思想，将处理一条指令所需要的不同步骤实现为独立的阶段。这个设计中，在任何时刻，每个阶段都可以处理不同的指令。我们的五阶段处理器流水线更加实用。本章中处理器设计的控制逻辑是用一种称为 HCL 的简单硬件描述语言来描述的。用 HCL 写的硬件设计能够编译和链接到本书提供的模拟器中，还可以根据这些设计

生成 Verilog 描述，它适合合成到实际可以运行的硬件上去。

- 第 5 章：优化程序性能。在这一章里，我们介绍了许多提高代码性能的技术，主要思想就是让程序员通过使编译器能够生成更有效的机器代码来学习编写 C 代码。我们一开始介绍的是减少程序需要做的工作的变换，这些是在任何机器上写任何程序时都应该遵循的。然后讲的是增加生成的机器代码中指令级并行度的变换，因而提高了程序在现代“超标量”处理器上的性能。为了解释这些变换行之有效的原理，我们介绍了一个简单的操作模型，它描述了现代乱序处理器是如何工作的，然后给出了如何根据一个程序的图形化表示中的关键路径来测量一个程序可能的性能。你会惊讶于对 C 代码做一些简单的变换能给程序带来多大的速度提升。
- 第 6 章：存储器层次结构。对应用程序员来说，存储器系统是计算机系统中最直接可见的部分之一。到目前为止，读者一直认同这样一个存储器系统概念模型，认为它是一个有一致访问时间的线性数组。实际上，存储器系统是一个由不同容量、造价和访问时间的存储设备组成的层次结构。我们讲述不同类型的随机存取存储器(RAM)和只读存储器(ROM)，以及磁盘和固态硬盘^①的几何形状和组织构造。我们描述这些存储设备是如何放置在层次结构中的，讲述访问局部性是如何使这种层次结构成为可能的。我们通过一个独特的观点使这些理论具体化，那就是将存储器系统视为一个“存储器山”，山脊是时间局部性，而斜坡是空间局部性。最后，我们向读者阐述如何通过改善程序的时间局部性和空间局部性来提高应用程序的性能。
- 第 7 章：链接。本章讲述静态和动态链接，包括的概念有可重定位的和可执行的目标文件、符号解析、重定位、静态库、共享目标库、位置无关代码，以及库打桩。大多数讲述系统的书中都不讲链接，我们要讲述它是出于以下原因。第一，程序员遇到的最令人迷惑的问题中，有一些和链接时的小故障有关，尤其是对那些大型软件包来说。第二，链接器生成的目标文件是与一些像加载、虚拟内存和内存映射这样的概念相关的。
- 第 8 章：异常控制流。在本书的这个部分，我们通过介绍异常控制流(即除正常分支和过程调用以外的控制流的变化)的一般概念，打破单一程序的模型。我们给出存在于系统所有层次的异常控制流的例子，从底层的硬件异常和中断，到并发进程的上下文切换，到由于接收 Linux 信号引起的控制流突变，到 C 语言中破坏栈原则的非本地跳转。

在这一章，我们介绍进程的基本概念，进程是对一个正在执行的程序的一种抽象。读者会学习进程是如何工作的，以及如何在应用程序中创建和操纵进程。我们

① 直译应为固态驱动器，但固态硬盘一词已经被大家接受，所以沿用。——译者注

会展示应用程序员如何通过 Linux 系统调用来使用多个进程。学完本章之后，读者就能够编写带作业控制的 Linux shell 了。同时，这里也会向读者初步展示程序的并发执行会引起不确定的行为。

- 第 9 章：虚拟内存。我们讲述虚拟内存系统是希望读者对它是如何工作的以及它的特性有所了解。我们想让读者了解为什么不同的并发进程各自都有一个完全相同的地址范围，能共享某些页，而又独占另外一些页。我们还讲了一些管理和操纵虚拟内存的问题。特别地，我们讨论了存储分配操作，就像标准库的 `malloc` 和 `free` 操作。阐述这些内容是出于下面几个目的。它加强了这样一个概念，那就是虚拟内存空间只是一个字节数组，程序可以把它划分成不同的存储单元。它可以帮助读者理解当程序包含存储泄漏和非法指针引用等内存引用错误时的后果。最后，许多应用程序员编写自己的优化了的存储分配操作来满足应用程序的需要和特性。这一章比其他任何一章都更能展现将计算机系统硬件和软件结合起来阐述的优点。而传统的计算机体系结构和操作系统书籍都只讲述虚拟内存的某一方面。
- 第 10 章：系统级 I/O。我们讲述 Unix I/O 的基本概念，例如文件和描述符。我们描述如何共享文件，I/O 重定向是如何工作的，还有如何访问文件的元数据。我们还开发了一个健壮的带缓冲区的 I/O 包，可以正确处理一种称为 `short counts` 的奇特行为，也就是库函数只读取一部分的输入数据。我们阐述 C 的标准 I/O 库，以及它与 Linux I/O 的关系，重点谈到标准 I/O 的局限性，这些局限性使之不适合网络编程。总的来说，本章的主题是后面两章——网络 and 并发编程的基础。
- 第 11 章：网络编程。对编程而言，网络是非常有趣的 I/O 设备，它将许多我们前面文中学习的概念（比如进程、信号、字节顺序、内存映射和动态内存分配）联系在一起。网络程序还为下一章的主题——并发，提供了一个很令人信服的上下文。本章只是网络编程的一个很小的部分，使读者能够编写一个简单的 Web 服务器。我们还讲述位于所有网络程序底层的客户端-服务器模型。我们展现了一个程序员对 Internet 的观点，并且教读者如何用套接字接口来编写 Internet 客户端和服务端。最后，我们介绍超文本传输协议(HTTP)，并开发了一个简单的迭代式 Web 服务器。
- 第 12 章：并发编程。这一章以 Internet 服务器设计为例介绍了并发编程。我们比较对照了三种编写并发程序的基本机制（进程、I/O 多路复用和线程），并且展示如何用它们来建造并发 Internet 服务器。我们探讨了用 `P`、`V` 信号量操作来实现同步、线程安全和可重入、竞争条件以及死锁等的基本原则。对大多数服务器应用来说，写并发代码都是很关键的。我们还讲述了线程级编程的使用方法，用这种方法来表达应用程序中的并行性，使得程序在多核处理器上能执行得更快。使用所有的核解决同一个计算问题需要很小心谨慎地协调并发线程，既要保证正确性，又要争取获得高性能。

本版新增内容

本书的第1版于2003年出版，第2版在2011年出版。考虑到计算机技术发展如此迅速，这本书的内容还算是保持得很好。事实证明 Intel x86 的机器上运行 Linux(以及相关操作系统)，加上采用 C 语言编程，是一种能够涵盖当今许多系统的组合。然而，硬件技术、编译器和程序库接口的变化，以及很多教师教授这些内容的经验，都促使我们做了大量的修改。

第2版以来的最大整体变化是，我们的介绍从以 IA32 和 x86-64 为基础，转变为完全以 x86-64 为基础。这种重心的转移影响了很多章节的内容。下面列出一些明显的变化：

- 第1章。我们将第5章对 Amdahl 定理的讨论移到了本章。
- 第2章。读者和评论家的反馈是一致的，本章的一些内容有点令人不知所措。因此，我们澄清了一些知识点，用更加数学的方式来描述，使得这些内容更容易理解。这使得读者能先略过数学细节，获得高层次的总体概念，然后回过头来进行更细致深入的阅读。
- 第3章。我们将之前基于 IA32 和 x86-64 的表现形式转换为完全基于 x86-64，还更新了近期版本 GCC 产生的代码。其结果是大量的重写工作，包括修改了一些概念提出的顺序。同时，我们还首次介绍了对处理浮点数据的程序的机器级支持。由于历史原因，我们给出了一个网络旁注描述 IA32 机器码。
- 第4章。我们将之前基于 32 位架构的处理器设计修改为支持 64 位字和操作的设计。
- 第5章。我们更新了内容以反映最近几代 x86-64 处理器的性能。通过引入更多的功能单元和更复杂的控制逻辑，我们开发的基于程序数据流表示的程序性能模型，其性能预测变得比之前更加可靠。
- 第6章。我们对内容进行了更新，以反映更多的近期技术。
- 第7章。针对 x86-64，我们重写了本章，扩充了关于用 GOT 和 PLT 创建位置无关代码的讨论，新增了一节描述更加强大的链接技术，比如库打桩。
- 第8章。我们增加了对信号处理程序更细致的描述，包括异步信号安全的函数，编写信号处理程序的具体指导原则，以及用 `sigsuspend` 等待处理程序。
- 第9章。本章变化不大。
- 第10章。我们新增了一节说明文件和文件的层次结构，除此之外，本章的变化不大。
- 第11章。我们介绍了采用最新 `getaddrinfo` 和 `getnameinfo` 函数的、与协议无关和线程安全的网络编程，取代过时的、不可重入的 `gethostbyname` 和 `gethostbyaddr` 函数。

- 第 12 章。我们扩充了利用线程级并行性使得程序在多核机器上更快运行的内容。

此外，我们还增加和修改了很多练习题和家庭作业。

本书的起源

本书起源于 1998 年秋季，我们在卡内基-梅隆(CMU)大学开设的一门编号为 15-213 的介绍性课程：计算机系统导论(Introduction to Computer System, ICS)[14]。从那以后，每学期都开设了 ICS 这门课程，每学期有超过 400 名学生上课，这些学生从本科二年级到硕士研究生都有，所学专业也很广泛。这门课程是卡内基-梅隆大学计算机科学系(CS)以及电子和计算机工程系(ECE)所有本科生的必修课，也是 CS 和 ECE 大多数高级系统课程的先行必修课。

ICS 这门课程的宗旨是用一种不同的方式向学生介绍计算机。因为，我们的学生中几乎没有人有机会亲自去构造一个计算机系统。另一方面，大多数学生，甚至包括所有的计算机科学家和计算机工程师，也需要日常使用计算机和编写计算机程序。所以我们决定从程序员的角度来讲解系统，并采用这样的原则过滤要讲述的内容：我们只讨论那些影响用户级 C 语言程序的性能、正确性或实用性的主题。

比如，我们排除了诸如硬件加法器和总线设计这样的主题。虽然我们谈及了机器语言，但是重点并不在于如何手工编写汇编语言，而是关注 C 语言编译器是如何将 C 语言的结构翻译成机器代码的，包括编译器是如何翻译指针、循环、过程调用以及开关(switch)语句的。更进一步地，我们将更广泛和全盘地看待系统，包括硬件和系统软件，涵盖了包括链接、加载、进程、信号、性能优化、虚拟内存、I/O 以及网络与并发编程等在内的主题。

这种做法使得我们讲授 ICS 课程的方式对学生来讲既实用、具体，还能动手操作，同时也非常能调动学生的积极性。很快地，我们收到来自学生和教职工非常热烈而积极的反响，我们意识到卡内基-梅隆大学以外的其他人也可以从我们的方法中获益。因此，这本书从 ICS 课程的笔记中应运而生了，而现在我们对它做了修改，使之能够反映科学技术以及计算机系统实现中的变化和进步。

通过本书的多个版本和多种语言译本，ICS 和许多相似课程已经成为世界范围内数百所高校的计算机科学和计算机工程课程的一部分。

写给指导教师们：可以基于本书的课程

指导教师可以使用本书来讲授五种不同类型的系统课程(见图 2)。具体每门课程则有

赖于课程大纲的要求、个人喜好、学生的背景和能力。图中的课程从左往右越来越强调以程序员的角度来看待系统。以下是简单的描述。

- **ORG**：一门以非传统风格讲述传统主题的计算机组成原理课程。传统的主题包括逻辑设计、处理器体系结构、汇编语言和存储器系统，然而这里更多地强调了对程序员的影响。例如，要反过来考虑数据表示对 C 语言程序的数据类型和操作的影响。又例如，对汇编代码的讲解是基于 C 语言编译器产生的机器代码，而不是手工编写的汇编代码。
- **ORG+**：一门特别强调硬件对应用程序性能影响的 ORG 课程。和 ORG 课程相比，学生要更多地学习代码优化和改进 C 语言程序的内存性能。
- **ICS**：基本的 ICS 课程，旨在培养一类程序员，他们能够理解硬件、操作系统和编译系统对应用程序的性能和正确性的影响。和 ORG+ 课程的一个显著不同是，本课程不涉及低层次的处理器体系结构。相反，程序员只同现代乱序处理器的高级模型打交道。ICS 课程非常适合安排到一个 10 周的小学期，如果期望步调更从容一些，也可以延长到一个 15 周的学期。
- **ICS+**：在基本的 ICS 课程基础上，额外论述一些系统编程的问题，比如系统级 I/O、网络编程和并发编程。这是卡内基-梅隆大学的一门一学期时长的课程，会讲述本书中除了低级处理器体系结构以外的所有章节。
- **SP**：一门系统编程课程。和 ICS+ 课程相似，但是剔除了浮点和性能优化的内容，更加强调系统编程，包括进程控制、动态链接、系统级 I/O、网络编程和并发编程。指导教师可能会想从其他渠道对某些高级主题做些补充，比如守护进程(daemon)、终端控制和 Unix IPC(进程间通信)。

图 2 要表达的主要信息是本书给了学生和指导教师多种选择。如果你希望学生更多地

章号	主题	课程				
		ORG	ORG+	ICS	ICS+	SP
1	系统漫游	•	•	•	•	•
2	数据表示	•	•	•	•	⊙ (d)
3	机器语言	•	•	•	•	•
4	处理器体系结构	•	•			
5	代码优化		•	•	•	
6	存储器层次结构	⊙ (a)	•	•	•	⊙ (a)
7	链接			⊙ (c)	⊙ (c)	•
8	异常控制流			•	•	•
9	虚拟内存	⊙ (b)	•	•	•	•
10	系统级 I/O				•	•
11	网络编程				•	•
12	并发编程				•	•

图 2 五类基于本书的课程

注：符号⊙表示覆盖部分章节，其中：(a)只有硬件；(b)无动态存储分配；(c)无动态链接；(d)无浮点数。
ICS+是卡内基-梅隆的 15-213 课程。

了解低层次的处理器体系结构，那么通过 ORG 和 ORG+ 课程可以达到目的。另一方面，如果你想将当前的计算机组成原理课程转换成 ICS 或者 ICS+ 课程，但是又对突然做这样剧烈的变化感到担心，那么你可以逐步递增转向 ICS 课程。你可以从 OGR 课程开始，它以一种非传统的方式教授传统的问题。一旦你对这些内容感到驾轻就熟了，就可以转到 ORG+，最终转到 ICS。如果学生没有 C 语言的经验（比如他们只用 Java 编写过程序），你可以花几周的时间在 C 语言上，然后再讲述 ORG 或者 ICS 课程的内容。

最后，我们认为 ORG+ 和 SP 课程适合安排为两期（两个小学期或者两个学期）。或者你可以考虑按照一期 ICS 和一期 SP 的方式来教授 ICS+ 课程。

写给指导教师们：经过课堂验证的实验练习

ICS+ 课程在卡内基-梅隆大学得到了学生很高的评价。学生对这门课程的评价，中值分数一般为 5.0/5.0，平均分数一般为 4.6/5.0。学生们说这门课非常有趣，令人兴奋，主要就是因为相关的实验练习。这些实验练习可以从 CS:APP 的主页上获得。下面是本书提供的一些实验的示例。

- 数据实验。这个实验要求学生实现简单的逻辑和算术运算函数，但是只能使用一个非常有限的 C 语言子集。比如，只能用位级操作来计算一个数字的绝对值。这个实验可帮助学生了解 C 语言数据类型的位级表示，以及数据操作的位级行为。
- 二进制炸弹实验。二进制炸弹是一个作为目标代码文件提供给学生的程序。运行时，它提示用户输入 6 个不同的字符串。如果其中的任何一个不正确，炸弹就会“爆炸”，打印出一条错误消息，并且在一个打分服务器上记录事件日志。学生必须通过对程序反汇编和逆向工程来测定应该是哪 6 个串，从而解除各自炸弹的雷管。该实验能教会学生理解汇编语言，并且强制他们学习怎样使用调试器。
- 缓冲区溢出实验。它要求学生通过利用一个缓冲区溢出漏洞，来修改一个二进制可执行文件的运行时行为。这个实验可教会学生栈的原理，并让他们了解写那种易于遭受缓冲区溢出攻击的代码的危险性。
- 体系结构实验。第 4 章的几个家庭作业能够组合成一个实验作业，在实验中，学生修改处理器的 HCL 描述，增加新的指令，修改分支预测策略，或者增加、删除旁路路径和寄存器端口。修改后的处理器能够被模拟，并通过运行自动化测试检测出大多数可能的错误。这个实验使学生能够体验处理器设计中令人激动的部分，而不需要掌握逻辑设计和硬件描述语言的完整知识。
- 性能实验。学生必须优化应用程序的核心函数（比如卷积积分或矩阵转置）的性能。这个实验可非常清晰地表明高速缓存的特性，并带给学生低级程序优化的经验。

- cache 实验。这个实验类似于性能实验，学生编写一个通用高速缓存模拟器，并优化小型矩阵转置核心函数，以最小化对模拟的高速缓存的不命中次数。我们使用 Valgrind 为矩阵转置核心函数生成真实的地址访问记录。
- shell 实验。学生实现他们自己的带有作业控制的 Unix shell 程序，包括 Ctrl+C 和 Ctrl+Z 按键，fg、bg 和 jobs 命令。这是学生第一次接触并发，并且让他们对 Unix 的进程控制、信号和信号处理有清晰的了解。
- malloc 实验。学生实现他们自己的 malloc、free 和 realloc(可选)版本。这个实验可让学生们清晰地理解数据的布局和组织，并且要求他们评估时间和空间效率的各种权衡及折中。
- 代理实验。实现一个位于浏览器和万维网其他部分之间的并行 Web 代理。这个实验向学生们揭示了 Web 客户端和服务器的主题，并且把课程中的许多概念联系起来，比如字节排序、文件 I/O、进程控制、信号、信号处理、内存映射、套接字和并发。学生很高兴能够看到他们的程序在真实的 Web 浏览器和 Web 服务器之间起到的作用。

CS:APP 的教师手册中有对实验的详细讨论，还有关于下载支持软件的说明。

第 3 版的致谢

很荣幸在此感谢那些帮助我们完成本书第 3 版的人们。

我们要感谢卡内基-梅隆大学的同事们，他们已经教授了 ICS 课程多年，并提供了富有见解的反馈意见，给了我们极大的鼓励：Guy Blelloch、Roger Dannenberg、David Eckhardt、Franz Franchetti、Greg Ganger、Seth Goldstein、Khaled Harras、Greg Kesden、Bruce Maggs、Todd Mowry、Andreas Nowatzky、Frank Pfenning、Markus Pueschel 和 Anthony Rowe。David Winters 在安装和配置参考 Linux 机器方面给予了我们很大的帮助。

Jason Fritts(圣路易斯大学，St. Louis University)和 Cindy Norris(阿帕拉契州立大学，Appalachian State)对第 2 版提供了细致周密的评论。龚奕利(武汉大学，Wuhan University)翻译了中文版，并为其维护勘误，同时还贡献了一些错误报告。Godmar Back(弗吉尼亚理工大学，Virginia Tech)向我们介绍了异步信号安全以及与协议无关的网络编程，帮助我们显著提升了本书质量。

非常感谢目光敏锐的读者们，他们报告了第 2 版中的错误：Rami Ammari、Paul Anagnostopoulos、Lucas Bärenfänger、Godmar Back、Ji Bin、Sharbel Bousemaan、Richard Callahan、Seth Chaiken、Cheng Chen、Libo Chen、Tao Du、Pascal Garcia、Yili Gong、

Ronald Greenberg、Dorukhan Gülöz、Dong Han、Dominik Helm、Ronald Jones、Mustafa Kazdagli、Gordon Kindlmann、Sankar Krishnan、Kanak Kshetri、Junlin Lu、Qiang Luo、Sebastian Luy、Lei Ma、Ashwin Nanjappa、Gregoire Paradis、Jonas Pfenniger、Karl Pichotta、David Ramsey、Kaustabh Roy、David Selvaraj、Sankar Shanmugam、Dominique Smulkowska、Dag Sørbo、Michael Spear、Yu Tanaka、Steven Triccanowicz、Scott Wright、Waiki Wright、Han Xu、Zhengshan Yan、Firo Yang、Shuang Yang、John Ye、Taketo Yoshida、Yan Zhu 和 Michael Zink。

还要感谢对实验做出贡献的读者，他们是：Godmar Back(弗吉尼亚理工大学，Virginia Tech)、Taymon Beal(伍斯特理工学院，Worcester Polytechnic Institute)、Aran Clauson(西华盛顿大学，Western Washington University)、Cary Gray(威顿学院，Wheaton College)、Paul Haiduk(德州农机大学，West Texas A&M University)、Len Hamey(麦考瑞大学，Macquarie University)、Eddie Kohler(哈佛大学，Harvard)、Hugh Lauer(伍斯特理工学院，Worcester Polytechnic Institute)、Robert Marmorstein(朗沃德大学，Longwood University)和 James Riely(德保罗大学，DePaul University)。

再次感谢 Windfall 软件公司的 Paul Anagnostopoulos 在本书排版和先进的制作过程中所做的精湛工作。非常感谢 Paul 和他的优秀团队：Richard Camp(文字编辑)、Jennifer McClain(校对)、Laurel Muller(美术制作)以及 Ted Laux(索引制作)。Paul 甚至找出了我们对缩写 BSS 的起源描述中的一个错误，这个错误从第 1 版起一直没有被发现！

最后，我们要感谢 Prentice Hall 出版社的朋友们。Marcia Horton 和我们的编辑 Matt Goldstein 一直坚定不移地给予我们支持和鼓励，非常感谢他们。

第 2 版的致谢

我们深深地感谢那些帮助我们写出 CS:APP 第 2 版的人们。

首先，我们要感谢在卡内基-梅隆大学教授 ICS 课程的同事们，感谢你们见解深刻的反馈意见和鼓励：Guy Blelloch、Roger Dannenberg、David Eckhardt、Greg Ganger、Seth Goldstein、Greg Kesden、Bruce Maggs、Todd Mowry、Andreas Nowatzyk、Frank Pfennig 和 Markus Pueschel。

还要感谢报告第 1 版勘误的目光敏锐的读者们：Daniel Amelang、Rui Baptista、Quarup Barreirinhas、Michael Bombyk、Jörg Brauer、Jordan Brough、Yixin Cao、James Carroll、Rui Carvalho、Hyoung-Kee Choi、Al Davis、Grant Davis、Christian Dufour、Mao Fan、Tim Freeman、Inge Frick、Max Gebhardt、Jeff Goldblat、Thomas Gross、Anita Gupta、John Hampton、Hiep

Hong、Greg Israelsen、Ronald Jones、Haudy Kazemi、Brian Kell、Constantine Kousoulis、Sacha Krakowiak、Arun Krishnaswamy、Martin Kulas、Michael Li、Zeyang Li、Ricky Liu、Mario Lo Conte、Dirk Maas、Devon Macey、Carl Marcinik、Will Marrero、Simone Martins、Tao Men、Mark Morrissey、Venkata Naidu、Bhas Nalabothula、Thomas Niemann、Eric Peskin、David Po、Anne Rogers、John Ross、Michael Scott、Seiki、Ray Shih、Darren Shultz、Erik Silksen、Suryanto、Emil Tarazi、Nawanan Theera-Ampornpunt、Joe Trdinich、Michael Trigoboff、James Troup、Martin Vopatek、Alan West、Betsy Wolff、Tim Wong、James Woodruff、Scott Wright、Jackie Xiao、Guanpeng Xu、Qing Xu、Caren Yang、Yin Yongsheng、Wang Yuanxuan、Steven Zhang 和 Day Zhong。特别感谢 Inge Frick，他发现了我们加锁复制(lock-and-copy)例子中一个极不明显但很深刻的错误，还要特别感谢 Ricky Liu，他的校对水平真的很高。

我们 Intel 实验室的同事 Andrew Chien 和 Limor Fix 在本书的写作过程中一直非常支持。非常感谢 Steve Schlosser 提供了一些关于磁盘驱动器的总结描述，Casey Helfrich 和 Michael Ryan 安装并维护了新的 Core i7 机器。Michael Kozuch、Babu Pillai 和 Jason Campbell 对存储器系统性能、多核系统和能量墙问题提出了很有价值的见解。Phil Gibbons 和 Shimin Chen 跟我们分享了大量关于固态硬盘设计的专业知识。

我们还有机会邀请了 Wen-Mei Hwu、Markus Pueschel 和 Jiri Simsa 这样的高人给予了一些针对具体问题的意见和高层次的建议。James Hoe 帮助我们写了 Y86 处理器的 Verilog 描述，还完成了所有将设计合成到可运行的硬件上的工作。

非常感谢审阅本书草稿的同事们：James Archibald(百翰杨大学，Brigham Young University)、Richard Carver(乔治梅森大学，George Mason University)、Mirela Damian(维拉诺瓦大学，Villanova University)、Peter Dinda(西北大学)、John Fiore(坦普尔大学，Temple University)、Jason Fritts(圣路易斯大学，St. Louis University)、John Greiner(莱斯大学)、Brian Harvey(加州大学伯克利分校)、Don Heller(宾夕法尼亚州立大学)、Wei Chung Hsu(明尼苏达大学)、Michelle Hugue(马里兰大学)、Jeremy Johnson(德雷克塞尔大学，Drexel University)、Geoff Kuenning(哈维马德学院，Harvey Mudd College)、Ricky Liu、Sam Madden(麻省理工学院)、Fred Martin(马萨诸塞大学洛厄尔分校，University of Massachusetts, Lowell)、Abraham Matta(波士顿大学)、Markus Pueschel(卡内基-梅隆大学)、Norman Ramsey(塔夫茨大学，Tufts University)、Glenn Reinmann(加州大学洛杉矶分校)、Michela Taufer(特拉华大学，University of Delaware)和 Craig Zilles(伊利诺伊大学香槟分校)。

Windfall 软件公司的 Paul Anagnostopoulos 出色地完成了本书的排版，并领导了制作团队。非常感谢 Paul 和他超棒的团队：Rick Camp(文字编辑)、Joe Snowden(排版)、

MaryEllen N. Oliver(校对)、Laurel Muller(美术)和 Ted Laux(索引制作)。

最后,我们要感谢 Prentice Hall 出版社的朋友们。Marcia Horton 总是支持着我们。我们的编辑 Matt Goldstein 由始至终表现出了一流的领导才能。我们由衷地感谢他们的帮助、鼓励和真知灼见。

第 1 版的致谢

我们衷心地感谢那些给了我们中肯批评和鼓励的众多朋友及同事。特别感谢我们 15-213 课程的学生们,他们充满感染力的精力和热情鞭策我们前行。Nick Carter 和 Vinny Furia 无私地提供了他们的 malloc 程序包。

Guy Blelloch、Greg Kesden、Bruce Maggs 和 Todd Mowry 已教授此课多个学期,他们给了我们鼓励并帮助改进课程内容。Herb Derby 提供了早期的精神指导和鼓励。Allan Fisher、Garth Gibson、Thomas Gross、Satya、Peter Steenkiste 和 Hui Zhang 从一开始就鼓励我们开设这门课程。Garth 早期给的建议促使本书的工作得以开展,并且在 Allan Fisher 领导的小组的帮助下又细化和修订了本书的工作。Mark Stehlik 和 Peter Lee 提供了极大的支持,使得这些内容成为本科生课程的一部分。Greg Kesden 针对 ICS 在操作系统课程上的影响提供了有益的反馈意见。Greg Ganger 和 Jiri Schindler 提供了一些磁盘驱动的描述说明,并回答了我们关于现代磁盘的疑问。Tom Striker 向我们展示了存储器山的比喻。James Hoe 在处理器体系结构方面提出了很多有用的建议和反馈。

有一群特殊的学生极大地帮助我们发展了这门课程的内容,他们是 Khalil Amiri、Angela Demke Brown、Chris Colohan、Jason Crawford、Peter Dinda、Julio Lopez、Bruce Lowekamp、Jeff Pierce、Sanjay Rao、Balaji Sarpeshkar、Blake Scholl、Sanjit Sethia、Greg Steffan、Tiankai Tu、Kip Walker 和 Yinglian Xie。尤其是 Chris Colohan 建立了愉悦的氛围并持续到今天,还发明了传奇般的“二进制炸弹”,这是一个对教授机器语言代码和调试概念非常有用的工具。

Chris Bauer、Alan Cox、Peter Dinda、Sandhya Dwarkadis、John Greiner、Bruce Jacob、Barry Johnson、Don Heller、Bruce Lowekamp、Greg Morrisett、Brian Noble、Bobbie Othmer、Bill Pugh、Michael Scott、Mark Smotherman、Greg Steffan 和 Bob Wier 花费了大量时间阅读此书的早期草稿,并给予我们建议。特别感谢 Peter Dinda(西北大学)、John Greiner(莱茨大学)、Wei Hsu(明尼苏达大学)、Bruce Lowekamp(威廉 & 玛丽大学)、Bobbie Othmer(明尼苏达大学)、Michael Scott(罗彻斯特大学)和 Bob Wier(落基山学院)在教学中测试此书的试用版。同样特别感谢他们的学生们!

我们还要感谢 Prentice Hall 出版社的同事。感谢 Marcia Horton、Eric Frank 和 Harold Stone 不懈的支持和远见。Harold 还帮我们提供了对 RISC 和 CISC 处理器体系结构准确的历史观点。Jerry Ralya 有惊人的见识，并教会了我们很多如何写作的知识。

最后，我们衷心感谢伟大的技术作家 Brian Kernighan 以及后来的 W. Richard Stevens，他们向我们证明了技术书籍也能写得如此优美。

谢谢你们所有的人。

Randal E. Bryant

David R. O'Hallaron

于匹兹堡，宾夕法尼亚州